

Series: Host HTTPD Proxy to Docker: Modern Defense (Part 2 of 7)

Author: H. Brister Wooley

Published: 2026-07-13

Series: Host HTTPD Proxy to Docker: Modern Defense (2/7)

Published Link: <https://github.com/hbwooley/deep-thoughts-and-ramblings/blob/main/2026-07-13-host-httpd-docker-ai-era-part2.md>

In Part 1 we covered the architecture and threat model. Now we get our hands dirty: **the actual host HTTPD configuration** – every directive, every module, every rule – with the specific modern attack it kills.

This isn't a reference manual – it's a threat-model-driven configuration. If a directive doesn't map to a real attack technique (traditional or AI-generated), it doesn't make the cut.

The Host HTTPD Stack

Host OS: RHEL 9 / Rocky Linux 9 (FIPS mode enabled)

Apache: httpd-2.4.62 (from AppStream, pinned)

Modules: mod_ssl, mod_security2, mod_qos, mod_rewrite, mod_headers, mod_proxy, mod_proxy_http, mod_proxy_wstunnel, mod_session, mod_session_crypto, mod_ssl

Core idea: The host HTTPD runs as apache:apache (UID 48), SELinux httpd_t, with **no Docker socket access, no container orchestration privileges**. It's a pure edge proxy.

httpd.conf – The Edge Proxy Baseline

Network Binding: One IP, One Job

```
# VuL ID: V-214246 Rule ID: SV-214246r612240_rule Control ID: AS24-U1-000360
```

```
Listen 192.0.2.10:80
```

```
Listen 192.0.2.10:443 https
```

Directive: Listen 192.0.2.10:80 and Listen 192.0.2.10:443 https

Threat: Multi-homed host exposure + AI-guided interface enumeration

Why It Works: A production host typically has 3-5 interfaces: management (iDRAC/iLO), backup, application, DMZ, possibly a dedicated monitoring NIC. If Apache binds to 0.0.0.0 (the default), it listens on *all* of them. An attacker who compromises the management network (via vulnerable BMC, default creds, or supply chain) can now reach your web server directly – bypassing any network ACLs that only protect the application VLAN.

Why this matters now: LLM-powered recon tools now correlate: 1. Shodan/Censys data -> your host's interfaces 2. Cloud metadata APIs (if cloud) -> internal IPs 3. SSL cert transparency logs -> subdomains mapped to IPs 4. **Result:** The AI knows *exactly* which interface to hit for direct access

Binding to a single application IP (192.0.2.10 – TEST-NET per RFC 5737) means the kernel drops packets on other interfaces *before* Apache processes them. Zero CPU cycles wasted. Zero attack surface.

Quick check:

```
# Confirm bind address
ss -tlnp | grep :80
# LISTEN 0 511 192.0.2.10:80 0.0.0.0:* users:(("httpd",pid=1234,fd=4))

# Test from wrong interface (should timeout/connection refused)
curl --interface 192.168.1.50 http://192.0.2.10/
# curl: (7) Failed to connect to 192.0.2.10 port 80: Connection refused
```

Process Isolation: Drop Privileges Immediately

User `apache`
Group `apache`

Directive: User `apache` / Group `apache`

Threat: Container escape -> host root + AI-generated privilege escalation chains

Why It Works: The master process starts as root (to bind ports 80/443), then immediately drops to `apache:apache` (UID 48/GID 48) before forking workers. All request handling happens as non-root.

Why this matters now: Modern exploit chains (often LLM-assisted) combine: 1. Memory corruption in HTTPD module (CVE-2024-XXXX) 2. Local privilege escalation (CVE-2024-YYYY, dirty pipe, io_uring, etc.) 3. **Result:** Root shell on host

If HTTPD runs as `apache`, step 1 gives you `apache` shell. Step 2 *still works* – but you're now attacking from a non-privileged context. The kernel enforces DAC/MAC/SELinux. You can't:

- Read /etc/shadow (DAC)
- Load kernel modules (capability)
- Access Docker socket (SELinux httpd_t vs container_runtime_t)
- Modify HTTPD binary/config (file ownership)

Quick check:

```
ps aux | grep httpd | head -1
# apache 1234 0.0 0.2 123456 7890 ? Ss 10:00 0:00 /usr/sbin/httpd -
DFOREGROUND

# Confirm no root workers
ps aux | grep httpd | grep -v apache
# (should return nothing)
```

Directory Hardening: Default Deny at the Edge

```
<Directory />
  Options FollowSymLinks
  Options -Indexes -Includes
  AllowOverride None
  Require all denied
</Directory>

<Directory "/var/www">
  <LimitExcept GET POST HEAD>
    Require all denied
  </LimitExcept>
  AllowOverride None
  Require all granted
</Directory>

<Directory "/var/www/html">
  <LimitExcept GET POST HEAD>
    Require all denied
  </LimitExcept>
  Options -Indexes +FollowSymLinks
  AllowOverride None
  Require all granted
</Directory>
```

Directive Breakdown:

Directive	Threat	Modern Context
Options -Indexes	Directory listing -> backup files, .git, configs	AI recon enumerates directory structures to find forgotten artifacts
Options -Includes	SSI injection (<!-- #exec cmd="..."	LLM generates SSI payloads for legacy apps; blocked at config level

Directive	Threat	Modern Context
AllowOverride None	-->) .htaccess override via uploaded files	AI crafts polymorphic webshells that write .htaccess to bypass auth
<LimitExcept GET POST HEAD>	Verb tampering (PUT, DELETE, TRACE, OPTIONS, PATCH, CONNECT)	AI fuzzing discovers non-standard verbs that bypass app logic
Require all denied (root)	Path traversal bypass	AI generates encoded/traversal payloads (.%2f, ..%00, Unicode variants)

Why Three Tiers? 1. **Root (/)** – Absolute lockout. Even if DocumentRoot is misconfigured, filesystem root is denied. 2. **/var/www** – Application root. Only safe verbs. No .htaccess (config is in httpd.conf, git-tracked). 3. **/var/www/html** – Static content. Same verbs, allows FollowSymLinks for legitimate deployment symlinks.

Quick check:

```
# Test directory listing blocked
curl -k https://192.0.2.10/some/path/ | grep -c "Index of"
# 0

# Test PUT blocked
curl -X PUT -k https://192.0.2.10/test.txt -d "data"
# 403 Forbidden

# Test TRACE blocked
curl -X TRACE -k https://192.0.2.10/
# 403 Forbidden
```

File Access Control: The 30+ Extension Blocklist

```
<Files ".ht*">
    Require all denied
</Files>

<FilesMatch
"\.(bak|config|dist|fla|inc|ini|log|psd|sh|sql|swp|htaccess|htpasswd|env|conf
ig|.js|md|gitignore|gitattributes|lock|yaml|editorconfig|org|asp|aspx|java|php
|mp3|txt|jsp|xhtml|git)$|~$">
    Require all denied
</FilesMatch>
```

Directive: <FilesMatch> with 30+ extensions + tilde

Threat: Accidental secret exposure + AI-powered artifact discovery

Why It Works: Developers make mistakes. Always. A .env with database creds gets committed. A backup.sql with production data gets left in the web root. A .git folder gets deployed. A config.php.bak from a quick edit.

Why this matters now: LLM-powered tools now: 1. Crawl your site -> find /backup-2024.sql -> extract creds -> pivot to DB 2. Scan for .git/config -> reconstruct repo history -> find hardcoded secrets 3. Identify .env.example -> infer production variable names -> targeted guessing

The <FilesMatch> is your **safety net**. Apache returns 403 *before* the request reaches your application. The attacker gets nothing. Your logs show the attempt (alerting).

Quick check:

```
# Test .env blocked
curl -k https://192.0.2.10/.env
# 403 Forbidden

# Test .git blocked
curl -k https://192.0.2.10/.git/config
# 403 Forbidden

# Test backup file blocked
curl -k https://192.0.2.10/backup.sql
# 403 Forbidden
```

Server Information Leakage: Shut Up Completely

ServerTokens **Prod**
ServerSignature **Off**
Header **unset** X-Powered-By
Header **unset** ETag
FileETag **None**
HostnameLookups **Off**
UseCanonicalName **Off**
TraceEnable Off

Directive	Threat	AI-Era Context
ServerTokens Prod	Header fingerprinting -> version-specific exploits	AI correlates Server: Apache/2.4.62 (Ubuntu) -> CVE database -> automated exploit selection
ServerSignature Off	Error page version	AI scrapes 404/500 pages for version

Directive	Threat	AI-Era Context
Header unset X-Powered-By	leakage Framework fingerprinting (PHP, Servlet, ASP.NET)	strings, build dates, module lists AI identifies X-Powered-By: PHP/8.2.1 -> targets PHP-specific RCE chains
FileETag None	Inode enumeration via ETag	AI uses inode numbers to map filesystem structure, find hidden files
HostnameLookups Off	Reverse DNS DoS + logging pollution	AI triggers massive PTR queries by hitting endpoints rapidly
UseCanonicalName Off	Host header injection -> redirect poisoning	AI crafts Host: evil.com -> password reset links point to attacker
TraceEnable Off	Cross-site tracing (XST) -> HttpOnly cookie theft	AI combines XST with XSS for session hijacking

Quick check:

Check headers

```
curl -k -I https://192.0.2.10/ | grep -i server
```

Server: Apache

No X-Powered-By

```
curl -k -I https://192.0.2.10/ | grep -i x-powered-by
```

(nothing)

No ETag with inode

```
curl -k -I https://192.0.2.10/ | grep -i etag
```

ETag: "abc123-def456" (no inode, just hash)

TRACE blocked

```
curl -X TRACE -k https://192.0.2.10/
```

403 Forbidden

HTTP Security Headers: Client-Side Defense in Depth

Header always set X-XSS-Protection "1; mode=block"

Header always set X-Frame-Options SAMEORIGIN

Header always set Strict-Transport-Security "max-age=31536000; includeSubDomains; preload"

Header always set X-Content-Type-Options "nosniff"

Header always set Referrer-Policy "no-referrer-when-downgrade"

Header always set Cache-Control "no-store, no-cache, must-revalidate"

Header always set Permissions-Policy "accelerometer=(), camera=(),

```
geolocation=(self), fullscreen=(self), microphone=(), payment=(), usb=(),  
web-share=())"
```

```
Header always edit Set-Cookie ^(.*)$ $1;HttpOnly;Secure;SameSite=strict
```

Header	Threat	AI-Era Context
X-XSS-Protection	Reflected XSS in older browsers	Defense in depth – AI generates polyglot payloads that bypass single layers
X-Frame-Options: SAMEORIGIN	Clickjacking	AI generates invisible iframe overlays for credential harvesting
HSTS: max-age=31536000; includeSubDomains; preload	SSL stripping, MITM, downgrade	AI automates HTTPS->HTTP downgrade attacks on public WiFi, corporate networks
X-Content-Type-Options: nosniff	MIME confusion (uploaded .jpg served as text/html)	AI crafts polyglot files that are valid image + valid HTML/JS
Referrer-Policy	Referrer leakage on HTTPS->HTTP	AI harvests referrers containing tokens, session IDs, PII from navigation
Cache-Control: no-store	Sensitive page caching	AI exploits shared computers, browser caches, proxy caches for data theft
Permissions-Policy	Browser API abuse (camera, mic, USB, payment)	AI injects malicious scripts that request permissions; browser refuses
Set-Cookie edit	Cookie theft via XSS, CSRF, network sniffing	HttpOnly (no JS), Secure (HTTPS only), SameSite=strict (no cross-site)

The always Keyword Matters: Without always, headers are *not* set on error responses (4xx, 5xx). AI triggers errors deliberately to strip protections. always = headers on every response.

Quick check:

```
curl -k -I https://192.0.2.10/ | grep -i -E "x-xss|x-frame|strict-  
transport|x-content|referrer|cache-control|permissions-policy|set-cookie"
```

Content Security Policy: Default-Deny, Explicit-Allow

```
Header always set Content-Security-Policy "default-src 'self'  
https://chat.example.org; \  
script-src 'self' 'unsafe-inline' 'unsafe-eval' https://www.google.com  
https://www.gstatic.com https://apis.google.com; \  
style-src 'self' 'unsafe-inline' https://chat.example.org"
```

```

https://fonts.googleapis.com; \
connect-src 'self' https://www.google.com https://chat.example.org
wss://angular.example.org:4000; \
img-src 'self' data: https://www.google.com https://www.gstatic.com
https://chat.example.org; \
frame-src 'self' https://www.google.com https://chat.example.org; \
frame-ancestors 'self' https://chat.example.org; \
font-src 'self' https://fonts.gstatic.com; \
base-uri 'self'; \
form-action 'self'; \
object-src 'none'; \
upgrade-insecure-requests"

```

CSP Directive Breakdown:

Directive	Value	Why This Specific Value
default-src 'self' https://chat.example.org	Baseline: only same-origin + chat widget	Forces explicit allowlist for every resource type
script-src	Self + Google APIs + 'unsafe-inline/eval' (legacy concession)	Goal: remove 'unsafe-inline' and 'unsafe-eval' – they're XSS enablers. Legacy code needs them; new code uses nonces.
style-src	Self + Google Fonts + chat widget	CSS from trusted CDNs only
connect-src	Self + Google + chat + WebSocket backend	fetch(), XMLHttpRequest, WebSocket, EventSource destinations
img-src	Self + data: + Google + chat	data: for inline icons/base64 images
frame-src	Self + Google + chat	What can be embedded in <iframe>
frame-ancestors	Self + chat widget	Who can embed <i>you</i> – prevents clickjacking
font-src	Self + Google Fonts	Web fonts only from trusted source
base-uri 'self'	Prevents <base	AI injects base tag to redirect all relative URLs

Directive	Value	Why This Specific Value
	href="evil.com"> tag injection	
form-action 'self'	Forms only submit to same origin	Prevents formjacking – AI modifies form action attribute
object-src 'none'	No <object>, <embed>, <applet>	Legacy plugin attack surface eliminated
upgrade-insecure-requests	Auto-upgrade HTTP->HTTPS	Defense in depth for any hardcoded HTTP links

Why Enumerate Every Third-Party? Because default-src 'self' blocks everything else. If you need Google Analytics, you *must* add it to connect-src and script-src. This forces you to audit every external dependency – a security win by itself.

Why this matters now: LLM-generated XSS payloads now: - Use eval(), Function(), setTimeout(string) – blocked by removing 'unsafe-eval' - Inject <script src="evil.com"> – blocked by script-src 'self' - Modify document.baseURI – blocked by base-uri 'self' - Submit forms to attacker – blocked by form-action 'self'

Quick check:

```
curl -k -I https://192.0.2.10/ | grep -i content-security-policy
# content-security-policy: default-src 'self' https://chat.example.org;
script-src ...

# Test CSP blocks inline script
curl -k "https://192.0.2.10/?q=<script>alert(1)</script>" -H "Accept:
text/html" | grep -c "alert"
# 0 (blocked or encoded)
```

Permissions Policy: Disable Browser Features You Don't Use

```
Header set Permissions-Policy: "accelerometer=(), camera=(),
geolocation=(self), \
fullscreen=(self), microphone=(), payment=(), usb=(), web-share=(), \
autoplay=(), encrypted-media=(), picture-in-picture=(), gyroscope=(), \
magnetometer=(), midi=(), otp-credentials=(), publickey-credentials-get=(), \
screen-wake-lock=(), serial=(), bluetooth=(), hid=(), gamepad=()"
```

Directive: Permissions-Policy (formerly Feature Policy)

Threat: XSS blast radius expansion via browser APIs

Why It Works: If an attacker achieves XSS, they can now access the user’s camera, microphone, USB devices, payment APIs, Bluetooth, gamepad – *unless* the browser refuses the API call.

Why this matters now: AI-generated XSS payloads now include: -
navigator.mediaDevices.getUserMedia({video: true}) -> webcam access -
navigator.usb.requestDevice() -> hardware key extraction -
navigator.credentials.get({publicKey: ...}) -> WebAuthn credential theft -
navigator.bluetooth.requestDevice() -> Bluetooth device enumeration

Permissions-Policy: camera=() means getUserMedia() throws SecurityError. The browser enforces it – no server-side logic needed.

Quick check:

```
curl -k -I https://192.0.2.10/ | grep -i permissions-policy
# permissions-policy: accelerometer=(), camera=(), ...
```

ssl.conf – TLS Termination That Survives AI-Generated Downgrades

Protocol & Cipher Configuration

```
SSLProtocol -ALL +TLSv1.2
SSLHonorCipherOrder On
SSLCipherSuite HIGH:!aNULL:!MD5:!3DES:!RC4:!CBC:!AES128:!AES256-
SHA:!CAMELLIA:!DHE-RSA-AES128:!DHE-RSA-AES256-SHA:!ECDHE-RSA-AES128:!ECDHE-
RSA-AES256-SHA384
```

Translation:

Setting	Meaning	AI-Era Threat Defeated
SSLProtocol -ALL +TLSv1.2	Disable everything, enable ONLY TLS 1.2	Protocol downgrade attacks – AI automates TLS 1.0/1.1 negotiation to exploit BEAST, POODLE, Lucky Thirteen
SSLHonorCipherOrder On	Server picks cipher, not client	Client-driven cipher negotiation – AI connects with weak cipher preference to force downgrade
SSLCipherSuite HIGH:!aNULL...	Start with HIGH, then explicitly remove: anonymous, MD5, 3DES, RC4,	Cipher suite downgrade – AI enumerates supported ciphers, finds weakest, forces negotiation

Setting	Meaning	AI-Era Threat Defeated
	ALL CBC , 128-bit AES, SHA-1, CAMELLIA, DHE/ECDHE with weak curves	

What’s Left? Only **AES-256-GCM** and **CHACHA20-POLY1305** – AEAD (Authenticated Encryption with Associated Data). No padding. No padding oracle. No CBC. No Lucky Thirteen.

Why No TLS 1.3? Compliance baseline mandates TLS 1.2. TLS 1.3 is cryptographically superior, but when the auditor checks for `SSLProtocol TLSv1.2`, you need exactly that. When the baseline updates, you change one line.

Why No CBC? Cipher Block Chaining mode is vulnerable to padding oracle attacks. AEAD ciphers authenticate *before* decrypting – no padding, no oracle, no Lucky Thirteen, no POODLE variants.

Quick check:

```
# Test TLS 1.1 rejected
openssl s_client -connect 192.0.2.10:443 -tls1_1 < /dev/null 2>&1 | grep -i "handshake failure"
# handshake failure

# Test TLS 1.2 accepted with AEAD
openssl s_client -connect 192.0.2.10:443 -tls1_2 < /dev/null 2>&1 | grep -i "cipher"
# Cipher      : TLS_AES_256_GCM_SHA384 (or AES256-GCM-SHA384)

# Verify no CBC ciphers
openssl s_client -connect 192.0.2.10:443 -tls1_2 -cipher 'AES256-SHA' < /dev/null 2>&1 | grep -i "handshake failure"
# handshake failure
```

Certificate Configuration

```
SSLCertificateFile /etc/pki/tls/certs/example.org.pubcert.with.chain.pem
SSLCertificateKeyFile /etc/pki/tls/private/example.org.privkey.pem
SSLCACertificateFile /etc/pki/ca-trust/source/anchors/all_certs.pem
```

File	Purpose	Permissions	AI-Era Context
SSLCertificateFile	Leaf cert + full chain (intermediates -	root:apache 644	AI scrapes cert transparency logs; full chain prevents “unknown issuer” errors that

File	Purpose	Permissions	AI-Era Context
	> root)		leak info
SSLCertificateKeyFile	Private key	root:apache 640	Critical: apache group can read (needed for TLS), world <i>cannot</i> . Even if attacker gets apache shell, they can't <i>replace</i> the key (root owns).
SSLCertificateFile	Trust store for client cert validation	root:apache 644	Contains enterprise CA roots. Used by SSLVerifyClient require for mutual TLS.

OCSP Stapling: Revocation Without the CA Round-Trip

```

SSLUseStapling On
SSLStaplingCache "shmcb:/run/httpd/ssl_stapling(512000)"
SSLStaplingStandardCacheTimeout 3600
SSLStaplingErrorCacheTimeout 600
SSLOCSPOverrideResponder http://ocsp.example-ca.com
SSLOCSPOverrideResponder On
SSLOCSPEnable On
SSLOCSPUseRequestNonce On

```

How It Works: 1. HTTPD periodically fetches signed OCSP response from CA 2. Caches it in shared memory (shmcb) – 512KB, volatile, survives restarts via cache 3. Staples it to TLS handshake – browser gets revocation status instantly 4. **No browser -> CA contact** = no visitor IP leak to CA, no CA downtime impact

Why this matters now: AI-powered tracking correlates: - Visitor IP + timestamp + target domain -> CA's OCSP logs - Builds browsing profiles across sites using same CA - **Stapling breaks this correlation** – CA never sees the visitor

Quick check:

```

# Check OCSP staple in handshake
openssl s_client -connect 192.0.2.10:443 -status < /dev/null 2>&1 | grep -A 10 "OCSP Response"
# OCSP Response Data:
#   OCSP Response Status: successful (0x0)
#   Cert Status: good

```

Client Certificate Authentication (The /cac Endpoint)

```

<Location /cac>
    SSLVerifyClient require
    SSLVerifyDepth 5
    SSLRequire %{SSL_CIPHER} !~ m/^(EXP|NULL)/ \

```

```

        and %{SSL_CLIENT_VERIFY} eq "SUCCESS" \
        and %{SSL_CLIENT_S_DN_O} eq "Example Organization"
    SSLUserName SSL_CLIENT_S_DN_CN
    RequestHeader set X-Forwarded-User %{SSL_CLIENT_S_DN_CN}e
    RequestHeader set X-Client-Cert %{SSL_CLIENT_CERT}e
    RedirectMatch 301 ^/cac/?(.*) /home
</Location>

```

Directive	Purpose	AI-Era Threat Defeated
SSLVerifyClient require	Mandates client cert; no cert = 403	Credential stuffing – attacker needs valid client cert from your PKI, not just password
SSLVerifyDepth 5	Max chain depth (leaf->intermediate->root)	Prevents excessively long chains that bypass validation logic
SSLRequire expression	Three conditions: (1) non-export/non-null cipher, (2) client verify SUCCESS, (3) Org DN = “Example Organization”	Supply chain / stolen cert – cert from another org (even valid) fails. AI-generated phishing can’t bypass PKI binding.
SSLUserName	Sets REMOTE_USER to cert CN	Downstream apps get authenticated identity without crypto code
RequestHeader set X-Forwarded-User	Passes CN to backend via header	Backend never sees cert – just username. Zero crypto bugs in app.
RequestHeader set X-Client-Cert	Passes full PEM cert to backend	Backend can do additional validation (revocation, extensions) if needed
RedirectMatch	After auth, redirect to app	/cac is <i>only</i> for cert handshake – not an application endpoint

Why This Architecture? The web server handles ALL PKI complexity: cert validation, chain building, revocation (OCSP/CRL), DN parsing, Org verification. Your application just reads X-Forwarded-User. **Zero crypto code in the app = zero crypto bugs in the app.**

Quick check:

```

# Test without client cert (should 403)
curl -k https://192.0.2.10/cac/

```

```
# 403 Forbidden
```

```
# Test with valid client cert (should 301 to /home)
```

```
curl -k --cert user.pem --key user.key https://192.0.2.10/cac/ -v 2>&1 | grep  
-E "301|Location"
```

```
# < Location: /home
```

mod_security.conf – ModSecurity WAF at the Edge

```
<IfModule security2_module>
```

```
    SecRuleEngine On
```

```
    SecRequestBodyAccess On
```

```
    SecRequestBodyLimit 13107200
```

```
    SecRequestBodyNoFilesLimit 13107200
```

```
    SecAuditEngine RelevantOnly
```

```
    SecAuditLogRelevantStatus "^(?:5|4(?:?!04))"
```

```
    SecAuditLogFormat JSON
```

```
    SecAuditLogType Concurrent
```

```
    SecAuditLogStorageDir /var/log/httpd/modsec_audit
```

```
    SecAuditLog /var/log/httpd/modsec_audit.log
```

```
    # OWASP Core Rule Set 3.3.4
```

```
    Include /etc/httpd/modsecurity.d/coreruleset-3.3.4/crs-setup.conf
```

```
    Include /etc/httpd/modsecurity.d/coreruleset-3.3.4/rules/*.conf
```

```
    # Local tuning -- YOUR rules, load LAST so they win
```

```
    Include /etc/httpd/modsecurity.d/local_rules/*.conf
```

```
</IfModule>
```

Setting	Purpose	AI-Era Context
SecRuleEngine On	Enables blocking. DetectionOnly = log only (tuning)	Never run DetectionOnly in production. AI generates bypasses for known rules; you need blocking.
SecRequestBodyAccess On	Inspects POST/PUT bodies (JSON, XML, form)	AI generates payloads in request bodies – JSON SQLi, XML XXE, prototype pollution. Without this, you only see headers/URL.
SecRequestBodyLimit 13107200	12.5 MB max request body	AI generates massive payloads to overwhelm parser (DoS via parser exhaustion)
SecAuditEngine RelevantOnly	Only logs transactions that trigger rules or errors (4xx/5xx except 404)	Log volume control – AI generates massive noise; 404s are 90%+ of traffic

Setting	Purpose	AI-Era Context
SecAuditLogFormat JSON	Structured logs for SIEM	AI-assisted log analysis needs structured data; regex on text logs fails
SecAuditLogType Concurrent	Each transaction = own file in StorageDir	No lock contention, no corruption, parallel processing
Include crs-setup.conf	CRS config: paranoia level, anomaly scoring, exclusions	Paranoia Level 2 = more rules, more false positives, more protection. Tune with local rules.
Include rules/*.conf	Loads 150+ CRS rules (SQLi, XSS, RFI, LFI, protocol anomalies, bots, scanners)	Generic protection against known attack patterns
Include local_rules/*.conf	YOUR overrides – load LAST so they win	Critical: Never edit CRS files directly. Local rules = your app-specific tuning, SecRuleRemoveById, custom rules.

Per-Location Exclusions: Surgical, Not Nuclear

```
# Large export endpoint -- internal, authenticated, rate-limited
<LocationMatch "^/server/api/scripts/export">
  <IfModule mod_security2.c>
    SecRuleEngine Off
    SecAuditEngine Off
  </IfModule>
</LocationMatch>

# Large file upload -- legitimate 16 GB uploads
<LocationMatch "^/server/api/core/upload">
  <IfModule mod_security2.c>
    SecRuleRemoveById 920350 949110 920420 980130 200003
    SecRequestBodyLimit 16106127360
  </IfModule>
</LocationMatch>
```

Why This Approach?

Endpoint	Approach	Rationale
metadata-export	SecRuleEngine Off	Generates huge XML/CSV. CRS triggers on XML entities, size, complexity. Instead of tuning 50 rules, turn off engine for <i>this one endpoint</i> . It's internal, authenticated, rate-limited – acceptable risk.

Endpoint	Approach	Rationale
bundles (upload)	SecRuleRemoveById (5 specific rules)	Legitimate 16 GB uploads. Don't turn off engine – surgically remove 5 rules that false-positive: 920350 (host header anomalies), 949110 (XSS in body), 920420 (SQLi in args), 980130 (RFI detection), 200003 (multipart boundary). Increase body limit <i>only</i> here.

Bottom line: Never disable SecRuleEngine globally. Never disable a rule globally. Scope exclusions to the exact LocationMatch that needs them.

Why this matters now: AI fuzzing tools now: 1. Discover your exclusions by probing endpoints 2. Target excluded endpoints with crafted payloads 3. **Mitigation:** Exclusions are scoped to *authenticated, internal, rate-limited* endpoints. Public endpoints have full CRS protection.

ModSecurity Local Rules: Your App-Specific Tuning (Load LAST, Win)

The production config includes **three critical exclusions** for DSpace endpoints that generate legitimate large payloads:

```
# In mod_security.conf -- surgical exclusions, not nuclear
SecRule REQUEST_URI "^/server/api/system/scripts/metadata-export"
"phase:1,id:100000,t:none,t:lowercase,pass,nolog,noauditlog,ctl:ruleEngine=off"
SecRule REQUEST_URI "^/server/api/system/scripts/metadata-import"
"phase:1,id:100001,t:none,t:lowercase,pass,nolog,noauditlog,ctl:ruleEngine=off"
SecRule REQUEST_URI "^/server/api/statistics/viewevents"
"phase:1,id:100002,t:none,t:lowercase,pass,nolog,noauditlog,ctl:ruleEngine=off"
```

Endpoint	Why Excluded	Risk Mitigation
metadata-export	Generates massive XML/CSV (100MB+)	Internal-only, authenticated, rate-limited
metadata-import	Accepts large XML payloads	Internal-only, authenticated, rate-limited
viewevents	Statistics dashboard, complex queries	Internal-only, authenticated

Bottom line: Never disable SecRuleEngine globally. Never disable a rule globally. Scope to exact LocationMatch. Load local rules LAST so they win.

Quick check:

```
# Confirm exclusions active
curl -k -X POST https://lib.example.org/server/api/system/scripts/metadata-export -H "Content-Type: application/xml" -d "<malicious>payload</malicious>"
# Should return 200 (ModSecurity bypassed for this endpoint only)

# Confirm OTHER endpoints still protected
curl -k -X POST https://lib.example.org/login -d "user='OR'1='1"
# 403 Forbidden (ModSecurity active)
```

mod_qos.conf – Rate Limiting That Survives AI Credential Stuffing

```
<IfModule qos_module>
  # Global connection limits
  QS_ClientEntries 100000
  QS_SrvMaxConnPerIP 50
  QS_SrvMaxConnCloseOnLimit 70

  # Per-URI limits (credential stuffing protection)
  QS_LocRequestLimitMatch "^(/login|/api/auth|/cac)" 10
  QS_LocRequestPerSecLimitMatch "^(/login|/api/auth|/cac)" 2

  # Request body limits
  QS_LocRequestLimitMatch "^/server/api/core/upload" 1000
  LimitRequestBody 13107200

  # SlowLoris / slow POST protection
  QS_RequestHeaderTimeout 30
  QS_RequestBodyTimeout 60
  QS_RequestBodyMinRate 500

  # Connection quality
  QS_CondLocRequestLimitMatch "^(/login|/api/auth|/cac)" 20
  QS_CondLocRequestPerSecLimitMatch "^(/login|/api/auth|/cac)" 5

  # Logging
  LogFormat "%h %l %u %t \"%r\" %>s %b \"%{Referer}i\" \"%{User-Agent}i\""
  QS_ClientIP=%{QS_ClientIP}e QS_Country=%{QS_Country}e" qos_combined
  CustomLog /var/log/httpd/qos_access_log qos_combined
</IfModule>
```

Directive	Threat	AI-Era Context
QS_SrvMaxConnPerIP 50	Connection exhaustion per IP	AI uses distributed botnets (1000s of IPs) – per-IP limit forces distribution
QS_LocRequestLimitMatch	Credential stuffing	AI enriches breached creds with

Directive	Threat	AI-Era Context
"^(/login /api/auth /cac)" 10	/ auth brute force	OSINT -> targeted spraying. 10 requests = fail fast.
QS_LocRequestPerSecLimitMatch	Rate limiting per second	AI uses "low and slow" to evade per-minute limits. Per-second catches it.
QS_RequestBodyMinRate 500	Slowloris / slow POST	AI sends 1 byte/second to hold workers. 500 B/s minimum = cut off.
QS_CondLocRequestLimitMatch	Conditional limits (after X requests, tighten)	Adaptive defense – first 20 requests normal, then tighten to 5/sec. AI learns and adapts; so does mod_qos.

Quick check:

```
# Test rate limit on login
for i in {1..15}; do curl -k -X POST https://192.0.2.10/login -d "user=test&pass=test"; done
# First 10: 200/401, Next 5: 429 Too Many Requests

# Test slowloris protection
# (requires specialized tool like slowhttptest)
slowhttptest -c 100 -H -g -o stats -i 10 -r 200 -t GET -u https://192.0.2.10/ -p 30
# Should show connections dropped after QS_RequestBodyMinRate enforcement
```

mod_qos v2: Production Values vs Blog Values

The production config has evolved stricter limits based on observed attack patterns:

```
# Production mod_qos.config (from
/production_config_files/httpd/conf.d/mod_qos.config)
QS_ClientEntries 100000
MaxClients 250
QS_SrvMaxConnPerIP 100          # Blog used 50 -- production allows more
legitimate connections
QS_ErrorResponseCode 429

# Trusted network exclusions (SSR, Docker, Localhost)
QS_ClientEventBlockExcludeIP 172.16.0.0/12
QS_ClientEventBlockExcludeIP 10.0.0.0/8
QS_ClientEventBlockExcludeIP 198.51.100.0/24
QS_ClientEventBlockExcludeIP 127.0.0.0/8

# VIP exemption for internal bypass header
QS_VipHeaderName X-DSpace-Internal-Bypass
SetEnvIf Remote_Addr "^172\.23\." QS_VipRequest=yes
SetEnvIf Remote_Addr "^10\.200\." QS_VipRequest=yes
SetEnvIf Remote_Addr "^127\." QS_VipRequest=yes
```

```

# Event definitions for granular control
SetEnvIf Request_URI "^/(server|handle)/" QS_Event
SetEnvIf Response_Code 404 is_404
SetEnvIf Response_Code 403 is_403
SetEnvIf Request_URI "^/server/api/discover/search" is_search
SetEnvIf Request_URI "^/server/api/core/bitstreams./*/content" is_bitstream
SetEnvIf Request_URI "^/server/api/core/(items|collections|communities)"
is_browse
SetEnvIfNoCase User-Agent "(python-
requests|curl|wget|scrapy|HTTrack|...|ClaudeBot|anthropic-ai|...)" bad_bot

# Level 1: Per-second burst
QS_LocRequestPerSecLimitMatch "^/server/api/discover/search" 50
QS_LocRequestPerSecLimitMatch "^/server/api/" 50

# Level 2: Short-term flood (conditional events)
QS_CondEventLimitCount is_404 20 300 3600      # 20 404s in 5 min -> 1 hr ban
QS_CondEventLimitCount is_403 15 300 3600      # 15 403s in 5 min -> 1 hr ban
QS_CondEventLimitCount bad_bot 10 60 86400      # 10 bad bot reqs in 1 min ->
24 hr ban
QS_CondEventLimitCount is_bitstream 60 60 900   # 60 downloads/min -> 15 min
ban
QS_CondEventLimitCount is_search 80 60 900      # 80 searches/min -> 15 min
ban

# Level 3: Global scraping protection
QS_ClientEventLimitCount 100 60 1800           # 100 reqs in 60 sec -> 30 min
ban

# Level 4: "Low and slow" scraper protection (NEW)
QS_CondEventLimitCount is_browse 1000 3600 86400 # 1000 browses/hr -> 24 hr
ban

```

Key Differences from Blog: | Setting | Blog (Part 2) | Production | Why Changed | |———|——
———|———|———| | QS_SrvMaxConnPerIP | 50 | 100 | Legitimate users behind
corporate NAT | | QS_LocRequestPerSecLimitMatch | /login | 2/sec | (uses event-based) |
Event-based adapts better | | bad_bot list | 14 UAs | 28+ UAs | Added ChatGPT-User,
GPTBot, Bytespider, CensysInspect, PerplexityBot, Omgilibot, SemrushBot, AhrefsBot,
MJ12bot, DotBot, Barkrowler, DataForSeoBot, SeznamBot | | is_browse limit | Not present |
1000/hr -> 24hr ban | Catches slow scrapers that evade per-min limits | Should show
connections dropped after QS_RequestBodyMinRate enforcement

fail2ban Integration: Dynamic IP Blocking at the Edge

```
### Jail Configuration (`/etc/fail2ban/jail.d/httpd-stig.conf`)
```

```
```ini
[httpd-auth]
enabled = true
port = http,https
filter = httpd-auth
logpath = /var/log/httpd/*_log
maxretry = 5
findtime = 300
bantime = 3600
action = iptables-allports[name=httpd-auth]
 httpd-banned-hosts[name=httpd-auth]

[httpd-modsec]
enabled = true
port = http,https
filter = httpd-modsec
logpath = /var/log/httpd/modsec_audit.log
maxretry = 10
findtime = 300
bantime = 86400
action = iptables-allports[name=httpd-modsec]
 httpd-banned-hosts[name=httpd-modsec]

[httpd-botsearch]
enabled = true
port = http,https
filter = httpd-botsearch
logpath = /var/log/httpd/*_log
maxretry = 20
findtime = 60
bantime = 86400
action = iptables-allports[name=httpd-botsearch]
 httpd-banned-hosts[name=httpd-botsearch]
```

### Action: httpd-banned-hosts (Atomic RewriteMap Update)

```
/etc/fail2ban/action.d/httpd-banned-hosts.conf
```

```
[Definition]
```

```
actionstart = touch /etc/httpd/conf.d/banned-hosts
 echo "RewriteMap hosts-deny txt:/etc/httpd/conf.d/banned-hosts"
> /etc/httpd/conf.d/banned-hosts.map
```

```
actionban = echo "<IP> 1" >> /etc/httpd/conf.d/banned-hosts.new
 mv /etc/httpd/conf.d/banned-hosts.new /etc/httpd/conf.d/banned-
hosts
 systemctl reload httpd
```

```
actionunban = grep -v "<IP> " /etc/httpd/conf.d/banned-hosts >
```

```
/etc/httpd/conf.d/banned-hosts.new
mv /etc/httpd/conf.d/banned-hosts.new /etc/httpd/conf.d/banned-
hosts
systemctl reload httpd
```

**Why Atomic mv?** fail2ban runs in parallel. If two jails ban/unban simultaneously, echo >> + reload causes race conditions – partial file read, HTTPD reload fails, legitimate traffic blocked. mv is atomic on POSIX.

### How It Works in HTTPD:

```
RewriteMap hosts-deny txt:/etc/httpd/conf.d/banned-hosts
RewriteCond ${hosts-deny:%{REMOTE_ADDR}|NOT-FOUND} !=NOT-FOUND [OR]
RewriteCond ${hosts-deny:%{REMOTE_HOST}|NOT-FOUND} !=NOT-FOUND
RewriteRule ^ /why-am-i-banned.html [L,PT]
```

**Why this matters now:** AI-powered credential stuffing: 1. Uses residential proxy networks (clean IPs) 2. Rotates IPs per request 3. **fail2ban + mod\_qos combo:** mod\_qos rate-limits per IP -> fail2ban bans IPs that trigger ModSecurity -> HTTPD blocks at edge via RewriteMap. Three layers, each catching what the others miss.

### Quick check:

```
Check banned hosts map
cat /etc/httpd/conf.d/banned-hosts
192.0.2.100 1
198.51.100.50 1

Test banned IP
curl -k --resolve "lib.example.org:443:192.0.2.100" https://lib.example.org/
302 -> /why-am-i-banned.html

Check fail2ban status
fail2ban-client status httpd-auth
Status for the jail: httpd-auth
|- Filter
| |- Currently failed: 2
| |- Total failed: 47
`-- Actions
|- Currently banned: 3
`-- Total banned: 156
```

---

## Rewrite Rules: Host Validation, Bot Blocking, Protocol Enforcement

```
AI scraper bots -- blocked BEFORE they hit ModSecurity
SetEnvIfNoCase User-Agent "(ChatGPT-User|GPTBot|ClaudeBot|anthropic-
ai|Amazonbot|ShapBot|Bytespider|CensysInspect|PerplexityBot|Omgilibot|Semrush
Bot|AhrefsBot|MJ12bot|DotBot|Barkrowler|DataForSeoBot|SeznamBot)" bad_bot_ua
```

```

RewriteCond %{ENV:bad_bot_ua} ^1$
RewriteRule .* - [F]

Host header validation -- only known hostnames allowed
RewriteCond %{HTTP_HOST}
!^(lib\.example\.org|localhost|127\.0\.0\.1|192\.0\.2\.10|10\.0\.0\.50)$ [NC]
RewriteCond %{HTTP_HOST} !^10\.0\.0\. [NC]
RewriteRule .? - [F]

Block TRACE/TRACK (XST)
RewriteCond %{REQUEST_METHOD} ^(TRACE|TRACK)
RewriteRule .* - [F]

Require HTTP/1.1 or 2.0 (no 1.0/0.9 -- smuggling vectors)
RewriteCond %{THE_REQUEST} !HTTP/(1\.1|2\.0)$
RewriteRule .* - [F]

Block dot files (/.git, /.env, /.aws, etc.)
RewriteCond %{SCRIPT_FILENAME} -d [OR]
RewriteCond %{SCRIPT_FILENAME} -f
RewriteRule "(\^|/)\." - [F]

Banned hosts map (fail2ban integration)
RewriteMap hosts-deny txt:/etc/httpd/conf.d/banned-hosts
RewriteCond ${hosts-deny:%{REMOTE_ADDR}|NOT-FOUND} !=NOT-FOUND [OR]
RewriteCond ${hosts-deny:%{REMOTE_HOST}|NOT-FOUND} !=NOT-FOUND
RewriteRule ^ /why-am-i-banned.html [L,PT]

```

Rule	Threat	AI-Era Context
<b>AI Bot UA blocking</b>	GPTBot, ClaudeBot, Bytespider, CensysInspect, etc.	AI companies scrape your content for training. SetEnvIfNoCase -> RewriteCond -> 403. Happens <i>before</i> ModSecurity, <i>before</i> proxy. Zero cost.
<b>Host header validation</b>	Host header injection, cache poisoning, vhost confusion	Only allows known hostnames. Host: evil.com -> 403. Prevents password reset poisoning, cache poisoning.
<b>TRACE/TRACK block</b>	Cross-site tracing (XST)	AI combines XST with XSS for HttpOnly cookie theft. Double-blocked (TraceEnable + Rewrite).
<b>HTTP/1.1+ only</b>	Protocol downgrade, request smuggling	HTTP/1.0 lacks Host header, chunked encoding – smuggling vectors. AI automates smuggling chains.
<b>Dot file blocking</b>	/.git/config, /.env, /.aws/credentials	AI recon finds exposed dot files in seconds. Blocked at edge.
<b>fail2ban RewriteMap</b>	Dynamic IP blocking	Atomic mv update -> instant block. No HTTPD restart needed (graceful reload).

---

## The takeaway

The host HTTPD is **not** a legacy component. It's your **most powerful security control** because:

1. **It sees everything first** – every packet hits host HTTPD before Docker
2. **It has full OS access** – iptables, SELinux, auditd, fail2ban, kernel modules
3. **It's immutable** – config in git, deployed via pipeline, verified on every run
4. **It's the choke point** – if host HTTPD blocks, the containers never see the attack

My strategy: 1. **Host HTTPD = Edge Policy Enforcement Point** – TLS, WAF, rate limiting, auth, bot blocking 2. **Docker Containers = Application Runtime** – non-root, read-only, minimal capabilities, mTLS 3. **Network Segmentation = Zero Trust** – frontend/backend networks, no container egress 4. **Monitoring = Continuous Verification** – structured logs, SIEM, nightly drift detection

---

## Next up

**Part 3:** The fail2ban deep-dive – dynamic IP blocking – Apache, Tomcat, Angular – hardened with non-root users, read-only filesystems, custom seccomp, SELinux MCS labels, mutual TLS from host proxy. The containers that *receive* the proxied traffic.

[Read Part 3 ->](#)

---

## Appendix: CompTIA Security+ SY0-701 CEU Mapping

*This appendix maps each technique in this post to CompTIA Security+ exam objectives for CEU submission.*

Technique Implemented	Security+ Domain	SY0-701 Objective	Rationale
Specific IP binding (Listen 192.0.2.10:443)	Domain 3: Security Architecture	3.4: Implement secure network architecture	Network segmentation, attack surface reduction
Non-root process (User apache)	Domain 3: Security Architecture	3.2: Implement secure system design	Least privilege, blast radius reduction
Directory default-deny (Require all denied)	Domain 3: Security Architecture	3.2: Implement secure system design	Zero trust filesystem access
HTTP method restriction (LimitExcept GET POST HEAD)	Domain 2: Security Architecture	2.3: Implement application security	Verb tampering prevention
File extension blocking (30+ extensions)	Domain 2: Security Architecture	2.3: Implement application security	Information disclosure prevention
ServerTokens Prod / ServerSignature Off	Domain 2: Security Architecture	2.3: Implement application security	Information leakage / fingerprinting prevention
Security headers (HSTS, CSP, X-Frame, Permissions-Policy)	Domain 2: Security Architecture	2.3: Implement application security	Client-side attack mitigation (XSS, clickjacking, SSL stripping)
TLS 1.2 only + AEAD ciphers (AES-256-GCM)	Domain 3: Security Architecture	3.5: Implement PKI/certificate management	Cryptographic protocol enforcement
OCSP Stapling	Domain 3: Security Architecture	3.5: Implement PKI/certificate management	Revocation checking without

Technique Implemented	Security+ Domain	SY0-701 Objective	Rationale
Client certificate authentication (mutual TLS)	Domain 3: Security Architecture	3.4: Implement secure network architecture	privacy leak PKI-based service authentication
ModSecurity CRS 3.3.4 + local rules	Domain 2: Security Architecture	2.4: Implement WAF / application-layer filtering	SQLi, XSS, RFI, LFI, protocol anomaly detection
mod_qos rate limiting (per-IP, per-URI)	Domain 4: Security Operations	4.5: Implement automation/orchestration	Credential stuffing, DoS prevention
fail2ban dynamic IP blocking (RewriteMap)	Domain 4: Security Operations	4.3: Automated response / intrusion prevention	Real-time threat response
Host header validation	Domain 2: Security Architecture	2.3: Implement application security	Host header injection, cache poisoning prevention
HTTP/1.1+ enforcement	Domain 2: Security Architecture	2.3: Implement application security	Request smuggling, protocol downgrade prevention
AI bot User-Agent blocking	Domain 1: Threats/Vulns/Mitigations	1.4: Analyze indicators of malicious activity	Automated scraping / reconnaissance prevention

**Primary Domain Coverage:** Domain 3 (Architecture & Engineering) – ~35% | Domain 2 (Architecture) – ~35% | Domain 4 (Operations) – ~20% | Domain 1 (Threats) – ~10%

**Total Security+ Mapping:** >=50% of content maps to SY0-701 objectives